

INTERNAL SESSION LOG

MTSAi Data Lake — Phase 2

Every step and command used to build, debug, and verify the export job

Companion document	MTSAi-Data-Lake-Build-Report.pdf (what was built and why)
This document	How it was built — the literal commands, in order
Repository	mtsai-datalake
Target account	Test — 690293068614 (ap-south-1)
Session date	15 September 2026

§ Contents

1.	Environment Setup
2.	Go Module & Dependencies
3.	Writing the Code
4.	Fixing the Already-Deployed IAM Role
5.	Local Integration Test Setup
6.	First Run & Debugging the DDL Syntax
7.	Debugging the Date Encoding Bug
8.	The Successful Run & Verification
9.	Container Build Check
10.	Cleanup & Commits

1 Environment Setup

Neither Go nor a running Docker daemon were available on this machine at the start of this session — both had to be installed/started before anything else was possible.

- 1 Installed Go via `winget` (no Go toolchain was present at all):

```
winget install --id GoLang.Go -e --accept-source-agreements --accept-package-agreements
```

```
Found Go Programming Language [GoLang.Go] Version 1.27.0
Downloading https://go.dev/dl/go1.27.0.windows-amd64.msi
Successfully installed
```

- 2 Docker Desktop's CLI was installed but the daemon wasn't running - started it and waited for the engine to come up:

```
" /c/Program Files/Docker/Docker/Docker Desktop.exe" &
# polled `docker version` until the Server section returned successfully
```

Go's install location wasn't yet on this shell's `PATH` (a fresh terminal would pick it up automatically; this session's existing shell needed it added manually for every subsequent command):

```
export PATH="/c/Program Files/Go/bin:$PATH"
```

2 Go Module & Dependencies

```
cd export
go mod init mtsai-datalake-export
```

Fetches every dependency the design called for, one `go get` per library:

```
go get github.com/jackc/pgx/v5           # Postgres driver, server-side cursor support
go get github.com/parquet-go/parquet-go   # Parquet writer
go get gopkg.in/yaml.v3                   # table-list config file
go get github.com/aws/aws-sdk-go-v2/config # AWS SDK v2
go get github.com/aws/aws-sdk-go-v2/service/s3
go get github.com/aws/aws-sdk-go-v2/service/athena
go get github.com/google/uuid             # run IDs
```

Before writing code against two libraries with real risk of getting the API wrong from memory (`parquet-go` 's dynamic/struct writer API, the AWS Athena SDK's exact struct field names), the actual downloaded source was read directly from the module cache rather than guessed:

```
go env GOMODCACHE
# -> C:\Users\Siby\go\pkg\mod
grep -n "^func NewGenericWriter\|^func WriteFile" ../parquet-go@v0.32.0/writer.go
grep -n "^type StartQueryExecutionInput" -A 20 ../athena@v1.66.0/api_op_StartQueryExecution.go
grep -n "^type QueryExecutionStatus" -A 20 ../athena@v1.66.0/types/types.go
```

This paid off later — it's exactly how the Parquet date-encoding bug in §7 got root-caused instead of guessed at.

After all the code in §3 was written:

```
go build ./...    # exit 0 - clean build
go vet ./...      # clean
go mod tidy       # resolves the full dependency graph, adds go.sum entries
gofmt -l .        # flagged 2 files -> gofmt -w to fix formatting
```

3 Writing the Code

Nine Go source files, one per concern, written in this order:

File	Responsibility
<code>internal/config/config.go</code>	Reads the fixed env-var contract Terraform injects, plus the YAML table-list config
<code>internal/config/tables.yaml</code>	The <code>trip_events</code> fixture table definition
<code>internal/model/tripevent.go</code>	The one typed row struct this v1 slice knows about
<code>internal/source/postgres.go</code>	<code>DECLARE CURSOR</code> / <code>FETCH FORWARD</code> streaming read from Postgres
<code>internal/parquetw/writer.go</code>	Rows → local Parquet file
<code>internal/lake/s3.go</code>	Upload to S3
<code>internal/lake/athena.go</code>	Run a SQL statement via Athena, poll until it finishes
<code>internal/lake/iceberg.go</code>	Orchestrates the staging-table → DELETE → INSERT → cleanup commit sequence
<code>internal/reconcile/reconcile.go</code>	Row-count + checksum comparison
<code>internal/manifest/manifest.go</code>	Writes the JSON manifest
<code>cmd/export/main.go</code>	Wires all of the above into one run

Plus `Dockerfile`, `docker-compose.test.yml`, `fixtures/trip_events.sql`, and `README.md` — see §5 for how those got used.

4 Fixing the Already-Deployed IAM Role

While designing the exact Athena/S3 calls the job makes, two permission gaps in the already-deployed `export-task` Terraform module became apparent — fixed before any testing, since the local test run uses admin credentials and wouldn't have caught them:

- No S3 permission to write Athena query results anywhere (`athena-results/export/*`)
- No `glue:DeleteTable` to clean up the staging table each run creates

```
# edited terraform/modules/export-task/main.tf, then:
cd terraform/envs/test
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan # Plan: 0 to add, 1 to change, 0 to destroy
terraform apply -auto-approve tfplan
```

5 Local Integration Test Setup

A local, Docker-based Postgres seeded with synthetic fixture data — matches the brief's own stated testing convention (Section 8: "integration tests... run against a Postgres container with the CITY_ZZ fixtures"). No real `mtsai-api` access was used or needed for this.

```
docker compose -f docker-compose.test.yml up -d
docker exec export-postgres-1 pg_isready -U export_test -d mtsai_export_test
docker exec export-postgres-1 psql -U export_test -d mtsai_export_test \
  -c "SELECT COUNT(*) FROM trip_events;" # -> 6 fixture rows
```

```
go build -o export.exe ./cmd/export
```

6 First Run & Debugging the DDL Syntax

The first real run got through Postgres and S3 cleanly, then failed the moment it tried to register the staging Parquet as a table in Athena:

```
export MTS_AI_DATALAKE_ENV="test"
export MTS_AI_DATALAKE_BUCKET="mtsai-datalake-test-690293068614-ap-south-1"
export MTS_AI_DATALAKE_RAW_DATABASE="test_raw"
export CONFIG_PATH="internal/config/tables.yaml"
export EXPORT_DATE="2026-01-15"
export
PGRES_CREDENTIALS='{ "host": "localhost", "port": 55432, "dbname": "mtsai_export_test", "username": "export_test", "password": "export_test" }'
./export.exe
```

```
read 5 row(s) from postgres for 2026-01-15
uploaded 5 row(s) to s3://.../raw/_staging/trip_events/<run_id>/data.parquet
export failed: commit to iceberg: create staging table: ...
InvalidRequestException: line 1:8: mismatched input 'EXTERNAL'.
sql: CREATE EXTERNAL TABLE IF NOT EXISTS ...
```

Rather than guess-and-rebuild the Go binary repeatedly, switched to testing SQL syntax directly against Athena via the CLI - much faster feedback per attempt.

Attempt 1 — the Trino-style form the error seemed to point toward:

```
aws athena start-query-execution \
  --query-string "CREATE TABLE ddl_test1 (trip_id bigint, city_code varchar) WITH (external_location = '...',
format = 'PARQUET')" \
  --query-execution-context Database=test_raw \
  --result-configuration "OutputLocation=s3://.../athena-results/export/"
```

```
InvalidRequestException: no viable alternative at input '... WITH ('
```

Rejected too — confirms the WITH(...) form is CTAS-only (needs AS SELECT).

Attempt 2 — plain Hive DDL, no EXTERNAL keyword:

```
aws athena start-query-execution \
  --query-string "CREATE TABLE ddl_test2 (trip_id bigint, city_code string) STORED AS PARQUET LOCATION '...'"
```

```
InvalidRequestException: External keyword required for table type HIVE
```

Attempt 3 — `EXTERNAL`, but without `IF NOT EXISTS` this time:

```
aws athena start-query-execution \
  --query-string "CREATE EXTERNAL TABLE ddl_test4 (id int) STORED AS PARQUET LOCATION '...'"
```

```
State: SUCCEEDED
```

This worked. Isolated the actual culprit: `IF NOT EXISTS` combined with `EXTERNAL` , not the `EXTERNAL` keyword itself.

Confirmed the full column list (including `string` / `date` / `timestamp` types) worked the same way, then applied the fix to `internal/lake/iceberg.go` (dropped `IF NOT EXISTS` ; each staging table name already has a unique run-ID suffix so it was never load-bearing) and to `cmd/export/main.go` (column types back to Hive's `string` , not Trino's `varchar`). Cleaned up every test table created along the way:

```
for t in ddl_test1 ddl_test2 ddl_test4 ddl_test5; do
  aws athena start-query-execution --query-string "DROP TABLE IF EXISTS $t" ...
done
```

7 Debugging the Date Encoding Bug

With the DDL syntax fixed, the staging table created successfully and a direct `SELECT` against it proved the mechanism worked - but surfaced a second, unrelated bug:

```
event_date: "-1454296-02-29" ← should be 2026-01-15
```

Rather than guess at a fix, went straight to the `parquet-go` source in the module cache:

```
grep -n "case reflect.TypeFor\[time.Time\]" .../parquet-go@v0.32.0/column_buffer_write.go
grep -n "func writeRowsFuncOfTime" -A 40 .../parquet-go@v0.32.0/column_buffer_write.go
```

Found it: the library's `time.Time` writer only branches correctly for the `Timestamp` logical type (physical `int64`); for `Date` (physical `int32`) it falls through to a default case that still writes a raw nanosecond `int64` into the `int32` column slot. Fixed by adding a Parquet-specific struct in `internal/parquetw/writer.go` that encodes the date as days-since-epoch (`int32`) by hand, converted from the model's `time.Time` at write time only:

```
func daysSinceEpoch(t time.Time) int32 {
    return int32(t.UTC().Truncate(24 * time.Hour).Unix() / 86400)
}
```

8 The Successful Run & Verification

One more gap before the full run succeeded: the `trip_events` Iceberg table itself had never actually been created in AWS (only a generic `test_table` from earlier Phase 1 testing existed) - created it by hand, per the brief's own Phase 1 step 4:

```
aws athena start-query-execution --query-string \  
"CREATE TABLE trip_events (trip_id bigint, vehicle_id_hash string, city_code string, \  
  event_date date, distance_km double, fare_amount double, created_at timestamp) \  
  LOCATION 's3://.../raw/trip_events/' TBLPROPERTIES ('table_type'='ICEBERG')"
```

```
go build -o export.exe ./cmd/export  
./export.exe
```

```
read 5 row(s) from postgres for 2026-01-15  
uploaded 5 row(s) to s3://.../raw/_staging/trip_events/<run_id>/data.parquet  
run <run_id> succeeded: 5 row(s), checksum=15, duration=10.25s
```

Independently verified rather than trusting the program's own exit code:

Check	Command	Result
Data actually landed, correct values/dates	<code>SELECT * FROM trip_events ORDER BY trip_id</code> via Athena	All 5 rows correct, event_date = 2026-01-15
Manifest correct	<code>aws s3 cp s3://.../export-manifests/2026-01-15/trip_events.json -</code>	row_count=5, checksum=15, success=true
Idempotency	Re-ran the binary for the same <code>EXPORT_DATE</code> , then <code>SELECT COUNT(*) ... WHERE event_date = DATE '2026-01-15'</code>	Still 5, not 10

9 Container Build Check

Confirmed the packaging works, even though nothing is pushed/deployed anywhere yet:

```
docker build --platform linux/arm64 -t mtsai-datalake-export:test .
```

```
go: go.mod requires go >= 1.27.0 (running go 1.23.12; GOTOOLCHAIN=local)  
ERROR: process "/bin/sh -c go mod download" did not complete successfully
```

The Dockerfile's base image (`golang:1.23-bookworm`) was older than what `go mod init` had recorded locally (1.27.0). Bumped the Dockerfile's base image tag to `golang:1.27-bookworm` and re-ran:

```
docker build --platform linux/arm64 -t mtsai-datalake-export:test .
```

```
=> exporting to image ... DONE
```

10 Cleanup & Commits

```
docker compose -f docker-compose.test.yml down
rm -f export.exe
docker rmi mtsai-datalake-export:test
```

Commit	Summary
589d223	Add Phase 2 export job core pipeline (v1 slice, trip_events only) + the two export-task IAM fixes
b60db9c	Fix three real bugs found by actually running the export job end-to-end (DDL syntax x2, date encoding)
fafc522	Update build report PDF with Phase 2 export job content

Net result

A working export pipeline for one table, verified with real data against the real Test AWS account — plus 5 previously-unknown bugs (3 in the new code's assumptions about Athena/parquet-go, 2 in already-deployed Terraform) that no amount of code review or `terraform plan` would have caught. All found and fixed within this one session by actually running things and reading source when a fix wasn't obvious.

Companion to [MTSAi-Data-Lake-Build-Report.pdf](#) . Reflects the literal sequence of commands run in this session on 15 September 2026 — condensed for readability where the same kind of command repeated (e.g. every Athena polling loop), not condensed where the exact sequence mattered (the DDL debugging in §6).